Bearcast Media
System Architecture

A full-size map of every actor, service and data flow behind bearcastmedia.com, redrawn from the October 2026 source code: the content pipeline, a page request from DNS to interactive, the radio and listener pipeline, the build-and-automation loop, and the security trust boundaries.

VERSION 2.0 · SUPERSEDES V1.0 (MAY 2026)

COMPANION TO

Technical White Paper v2.0
Cybersecurity White Paper v2.0

AUTHORED BY

Cam Garrison
Web Director

LAST UPDATED

October 2026
© 2026 Bearcast Media

Reading This Document

Node types

ACTOR	A person or role — editors, DJs, visitors, the Web Director.
EXTERNAL SERVICE	Third-party platforms: Sanity Content Lake, Radio.co, YouTube, iTunes, Slack, GitHub.
BEARCAST CODE	Code Bearcast owns: the site, page modules, Studio, Pages Functions.
EDGE / CDN	Geographically distributed delivery: Cloudflare, Sanity's API and image CDNs.
DATA / AUTOMATION	Dashed: drafts, caches, browser storage, GitHub Actions jobs.
STORE	Databases (Cloudflare D1).

Edge types

- Read / request — data flowing one way (GET, fetch)
- Write / mutation — publish, POST, webhook, git push
- Automation / triggered — builds, scheduled jobs
- Live audio — the Radio.co stream
- Gate passed — e.g. both CI checks green

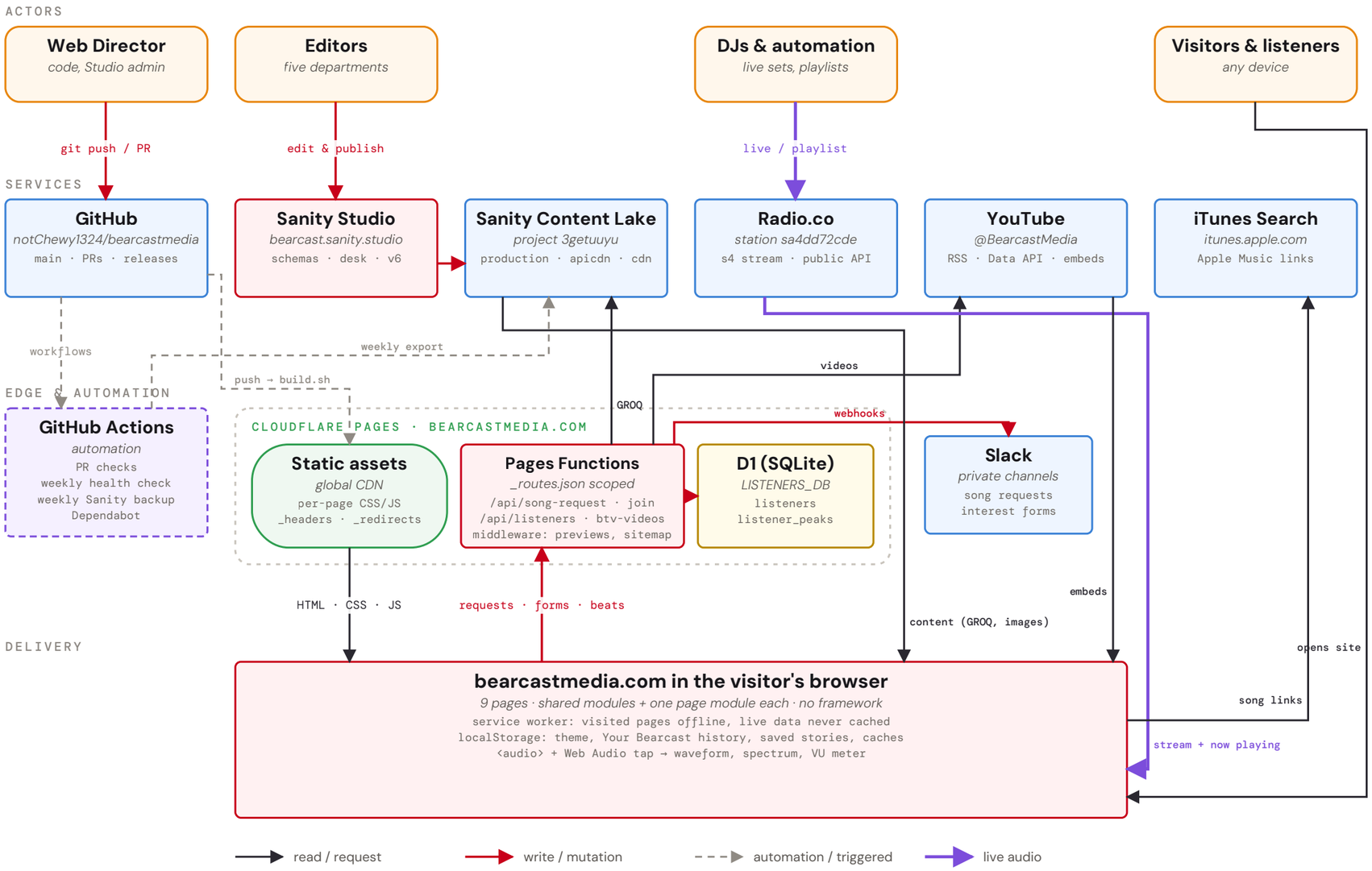
What changed since v1.0

- Pages Functions and a D1 database now sit beside the static files.
- Slack, YouTube and Apple's iTunes Search join the service list.
- The player is the site's own `<audio>` element, not the Radio.co widget.
- GitHub Actions check, monitor and back up the system.
- New diagrams: build & automation, and trust boundaries.

How to use these diagrams. Diagram 1 is the map. Diagrams 2–4 follow the three flows that matter most day to day — content, page requests and radio. Diagram 5 shows how code reaches production and how the system is watched. Diagram 6 numbers the trust boundaries used in the Cybersecurity White Paper. When something breaks, find the diagram for that flow and trace the arrows back from the symptom: the label on each arrow says what should be moving.

Diagram 1 — Full System Overview

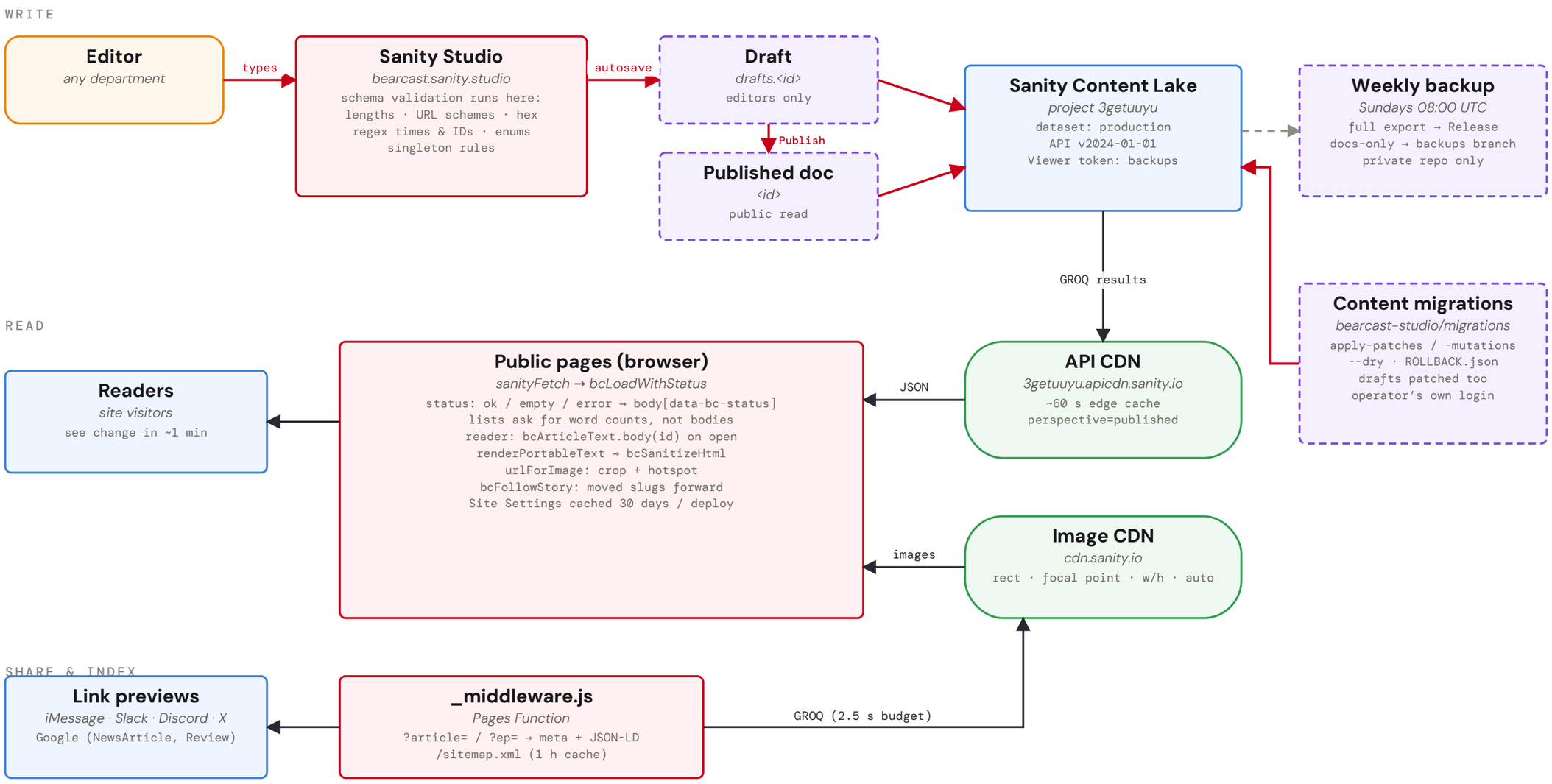
EVERY ACTOR, SERVICE AND FLOW



Top row: people. **Middle:** the services they use. **Green zone:** the Cloudflare Pages project — static files, five Functions scoped by `_routes.json`, and the D1 listener database. **Bottom:** the visitor's browser, where every flow converges. The browser talks directly to Sanity, Radio.co and iTunes (exactly the hosts in the CSP's `connect-src`); YouTube data and Slack are only reached through Functions.

Diagram 2 — Editorial Workflow

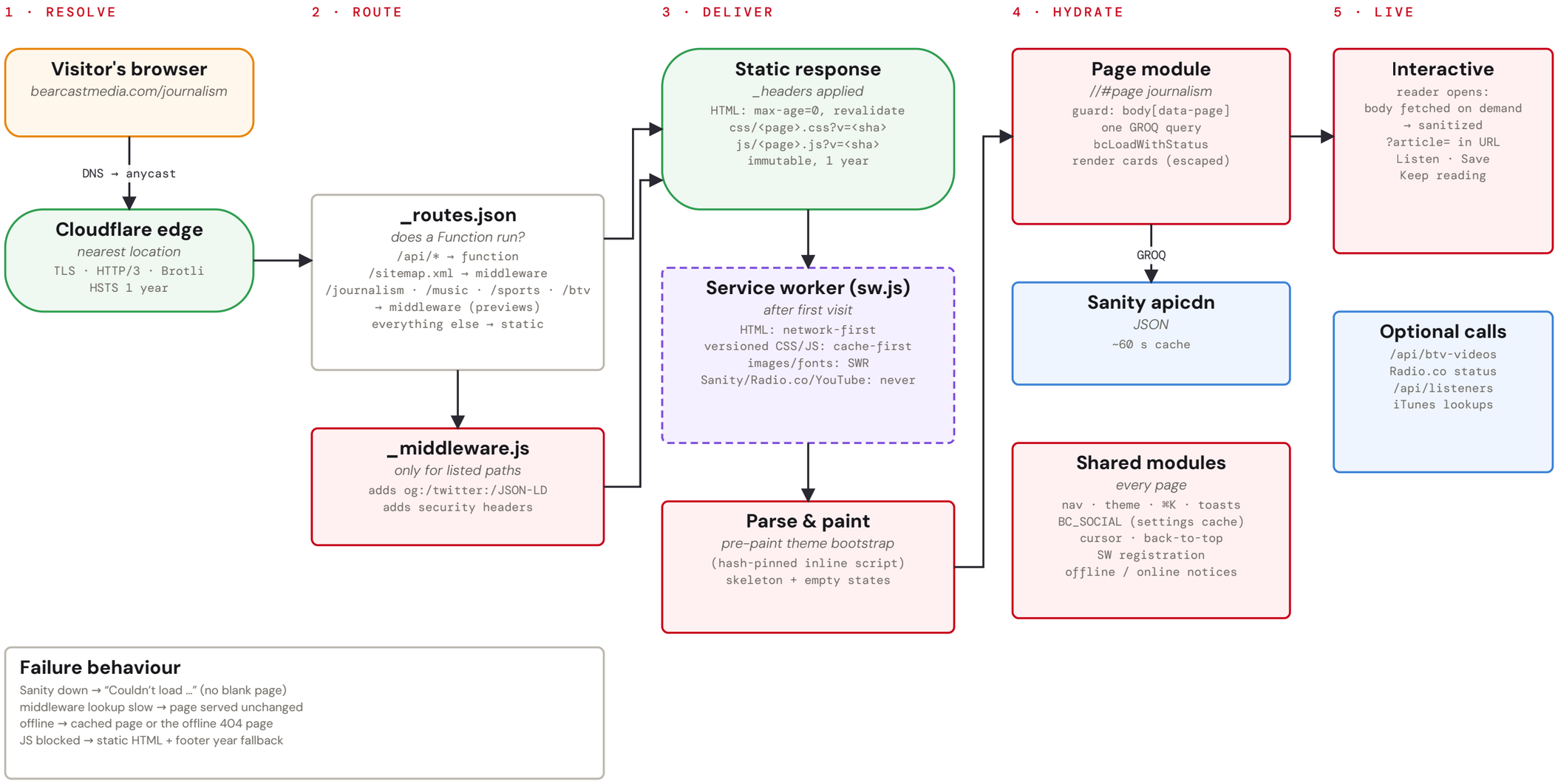
FROM KEYSTROKE TO READER



Write: the Studio validates as you type and autosaves to a draft that only editors can read. Publishing copies it to the public document. **Read:** pages query the API CDN (about a minute of caching, never drafts) and render with explicit loading / empty / error states. **Share & index:** for link-preview bots, the middleware looks the story up and rewrites the page head; `/sitemap.xml` lists every story. Backups and migrations work on the same dataset from the side.

Diagram 3 — Visitor Request Flow

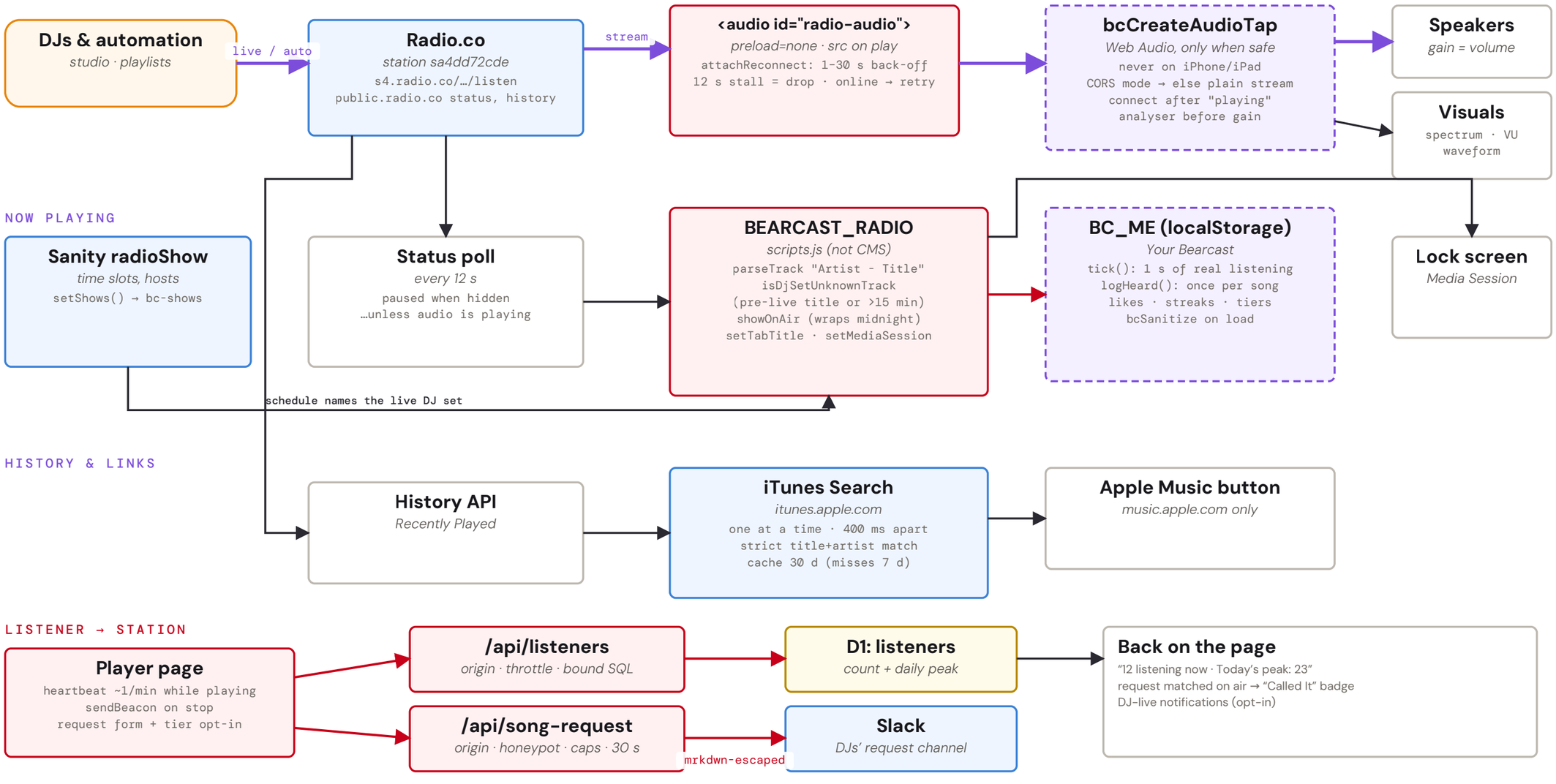
WHAT HAPPENS WHEN SOMEBODY OPENS A PAGE



The HTML, CSS and JS are the same for everyone and come from the nearest Cloudflare location. Only the paths listed in `_routes.json` touch a Function. Asset URLs carry the deploy's commit hash, so they are cached for a year and still never stale. After the first visit the service worker serves versioned assets from the device and keeps visited pages available offline. Content arrives in a second step, after the page is already visible.

Diagram 4 — Radio & Listener Pipeline

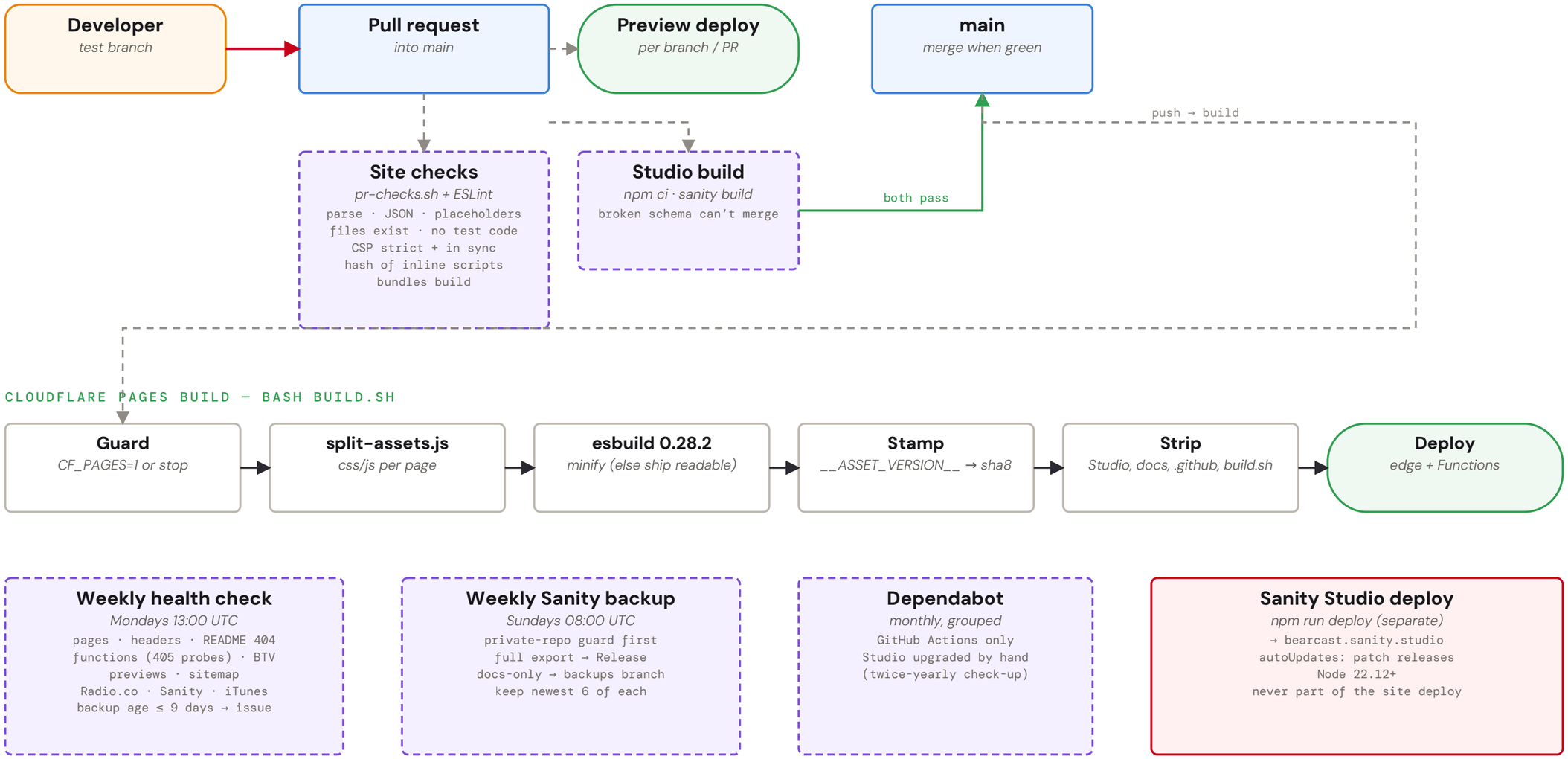
AUDIO FIRST, EVERYTHING ELSE OPTIONAL



Three streams meet in the player: **audio** (Radio.co → `<audio>`, optionally tapped by Web Audio for real visuals), **metadata** (status every 12 s and history, with stale-track detection during live sets and the scheduled show from Sanity naming the DJ), and **listener activity** (heartbeats to the D1 counter, requests to Slack, personal history in `localStorage`). If any visual or data path fails, the audio keeps playing.

Diagram 5 — Build, Deploy & Automation

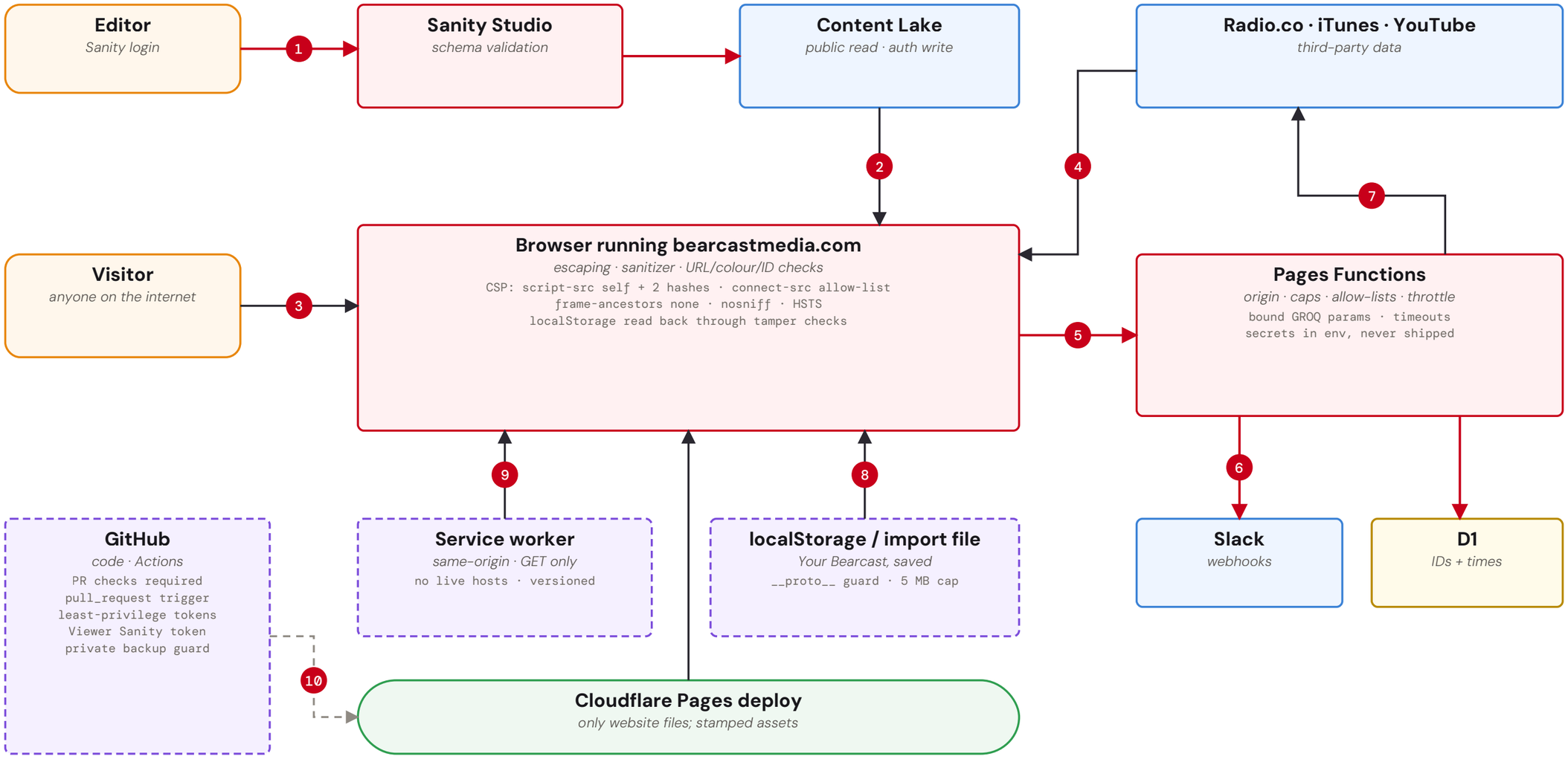
HOW CODE REACHES PRODUCTION AND IS WATCHED



Changes go through a test branch with its own preview deployment and a pull request that must pass both CI jobs. Merging to `main` triggers Cloudflare's build: per-page bundles, minification, version stamping and removal of everything that is not the website. Separately, GitHub Actions back up the CMS every Sunday, test the live site and its dependencies every Monday (opening an issue on failure), and Dependabot proposes Action updates monthly. The Studio deploys on its own.

Diagram 6 — Trust Boundaries

WHERE DATA CROSSES INTO SOMETHING MORE TRUSTED



1 Editor → Studio (login, validation) · **2** Content Lake → browser (encoding, sanitizer) · **3** site → visitor's browser (CSP, headers) · **4** third-party APIs → browser (escaping, URL/ID checks) · **5** visitor → Functions (origin, caps, allow-lists, throttle) · **6** Functions → Slack (secret webhooks, mrkdwn escaping) · **7** Functions → Sanity/YouTube (bound params, timeouts) · **8** browser storage and import files → page (tamper checks) · **9** service worker (same-origin, GET-only) · **10** GitHub → deploy and backups (required checks, least-privilege tokens). Numbering matches the Cybersecurity White Paper §02.

Reference — Browser Storage Map

EVERYTHING THE SITE KEEPS ON A VISITOR'S DEVICE

KEY	TYPE	WRITTEN BY	HOLDS	LIFETIME
`bc-theme`	local	theme module, head bootstrap	auto / light / dark	until changed
`bc_me_v1`	local	BC_ME (home + radio players)	Your Bearcast: days, plays, months, artists, likes, badges, tiers, profile	plays 60 d · days ~400 d · months forever
`bc_saved_v1`	local	bcReader	saved stories (≤200)	until removed
`bc_recent_q_v1` · `bc-cmdk-recent`	local	page search · ⌘K	last 5 searches per page · recent picks	rolling
`bc_site_settings_v1`	local	bcSiteSettings	Site Settings (social links)	30 days or next deploy
`bc_am_links`	local	radio module	Apple Music track URLs (≤400)	30 d hits · 7 d misses
`bc_pending_requests`	local	request form	last 5 requests for "Called It"	12 hours
`bc_viz_off` · `bc_viz_style`	local	audio tap · visualizer	real-audio opt-out flag · chosen style	1–30 days · until changed
`bc-volume` · `bc_you_period` · `bc_trophies_open` · `bc_aurora_welcomed`	local	radio module	UI preferences	until changed
`bc_dj_notify` · `bc_live_since`	local	radio module	DJ-live notification opt-in · live-set start time	until changed / set ends
`bc-cmdk-index-v3`	session	⌘K palette	archive titles & metadata (~45 KB)	browser session
`bc_listen_id`	session	BC_LISTENERS	random per-tab listener ID	tab session
Cache Storage `bc-static-v-<sha>`, `bc-pages-v-<sha>`	SW	sw.js	assets, visited pages, offline page	replaced every deploy

No cookies, no analytics, no third-party scripts. Server-side, the only per-visitor data is a random tab ID and a timestamp in D1 (pruned after ten minutes); interest forms and song requests are relayed to Slack, not stored by the site.

Reading These Diagrams Together

What the diagrams deliberately leave out

- **Vendor internals.** Sanity, Cloudflare, Radio.co, Slack and GitHub are black boxes; the diagrams show what goes in and comes out.
- **Code-level detail.** Queries, validation rules and function logic are quoted in the Technical and Cybersecurity White Papers.
- **Editorial process.** Who writes and approves what is in the Editor Handbook.

Where each setting lives

Station / stream	`BEARCAST_RADIO` in scripts.js
Sanity project & API version	scripts.js · sanity.config.ts · _middleware.js · health-check.sh
Headers & CSP	`_headers` + `SECURITY_HEADERS` (kept identical by CI)
Which paths run code	`_routes.json`
Secrets & D1 binding	Cloudflare Pages settings
Backup token & repo	GitHub Actions secrets and variables

Keeping this document current

Update the diagrams when the architecture changes — a new external service, Function, CSP origin or document type; changes to the build, the radio integration or the page set. Cosmetic changes don't need an update: the diagrams describe *what flows where*, not what it looks like.

The documentation set

- **Technical White Paper** — the architecture in prose, with the code.
- **Cybersecurity White Paper** — threat model, defenses, findings, verification.
- **Editor Handbook** — using the Studio day to day.
- **System Architecture Diagram** (this document) — the map.
- `MAINTENANCE.md` in the repository — the operating runbook.

Bearcast Media System Architecture Diagram · v2.0 · October 2026 · Authored by Cam Garrison · Drawn from the October 2026 source snapshot